

Microservices

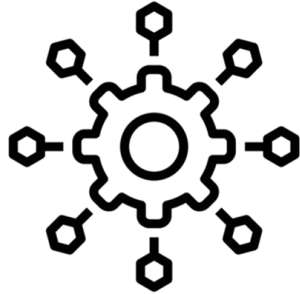
Microservicios



Universidad de
La Sabana

FACULTAD DE INGENIERÍA

Agenda



Microservices II

Anti-Patterns

- Anemic DDD
- Data Synchronization
- Shared Kernel (Abuse)

Methodology II: Assemblage

- Comparative with DDD
- Focus
- Pattern
- Method

API

- Design
- Gateway
 - Gateway Routing Pattern
 - Gateway Offloading Pattern
- Implementation Model

Migration Strategies

- Strangler Fig Pattern
- Anticorruption Layer Pattern
- Dapr

Additional Patterns

- Sagas
- Microfrontends

Topics to explore

Anemic Domain Model

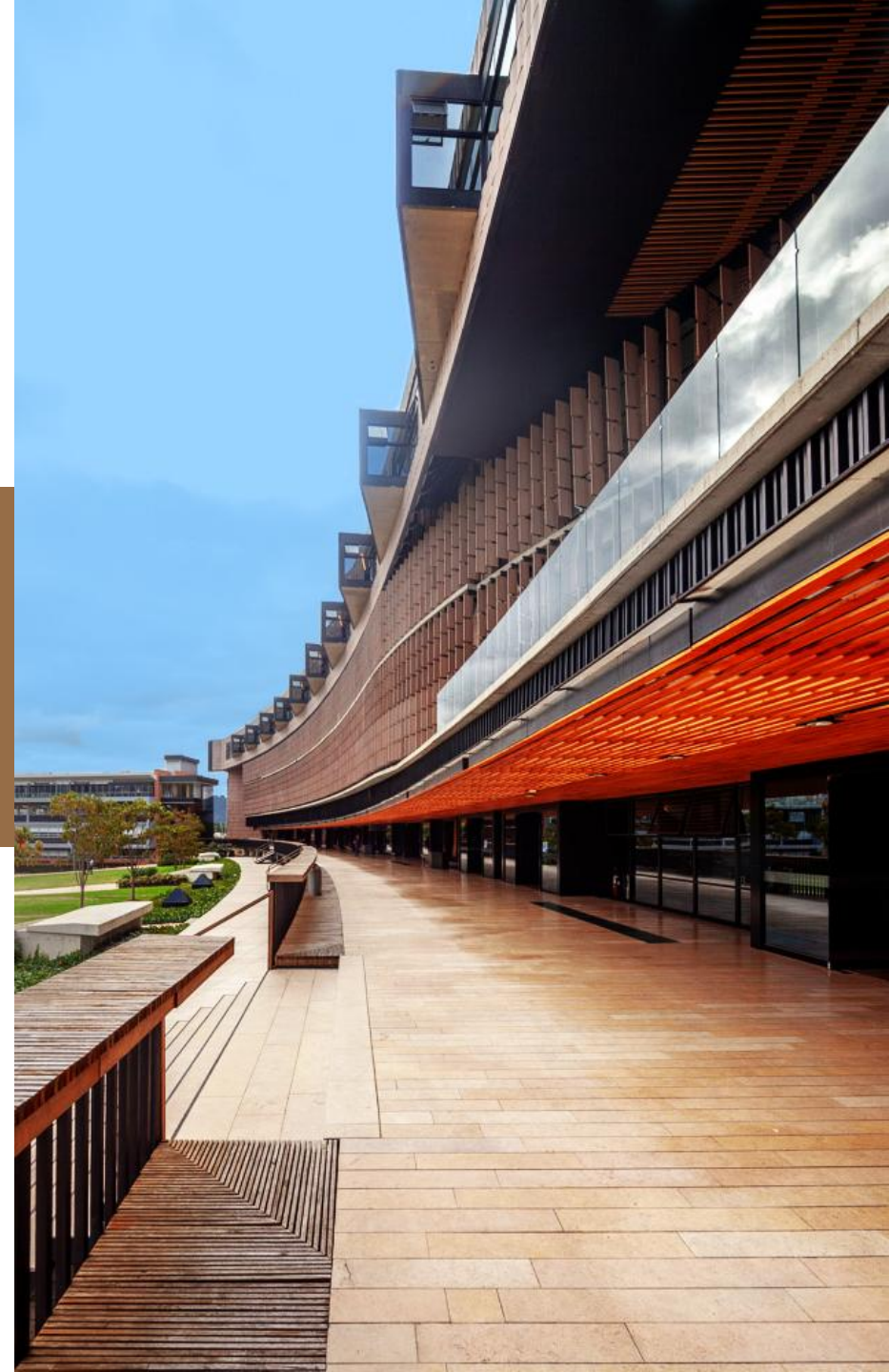
Data Synchronization

Shared Kernel



Universidad de
La Sabana

FACULTAD DE INGENIERÍA



Anemic Domain Model

You use technical instead of biz focus. It's a cognitive bias from Object-Oriented Domain or from your U, to use OOP.
Aggregates/Entities become public accessors (get/set) from data repositories. Those logic are outside on external components (Domain Services?, ESB?, Adapters?).

Reasons:

Cognitive Bias on Availability (ORM: MS Entity o Hibernate Framework).

Time-to-market

Purist Framework (a lot of concepts, framework).

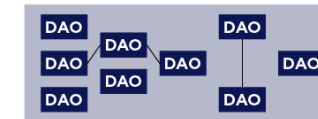
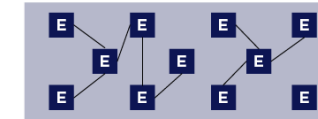
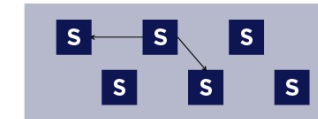
Solutions:

Invariants on Aggregates show business value (Encapsulate value, don't create nanoservices).

Don't fill all actions on Domain Services.

Don't create dependencies on infra.

ANEMIC DOMAIN MODEL



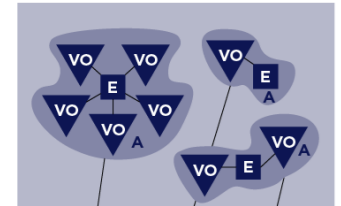
S SERVICE

E ENTITY

R REPOSITORY

A AGGREGATE

DDD

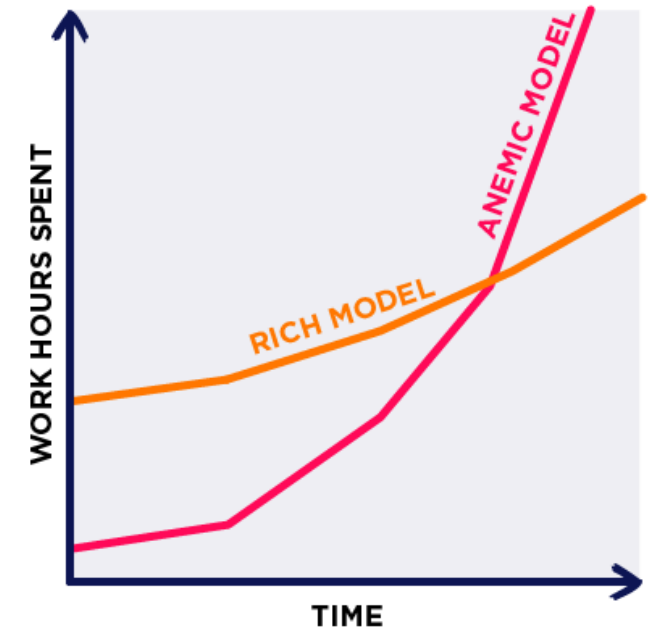


VO VALUE OBJECT

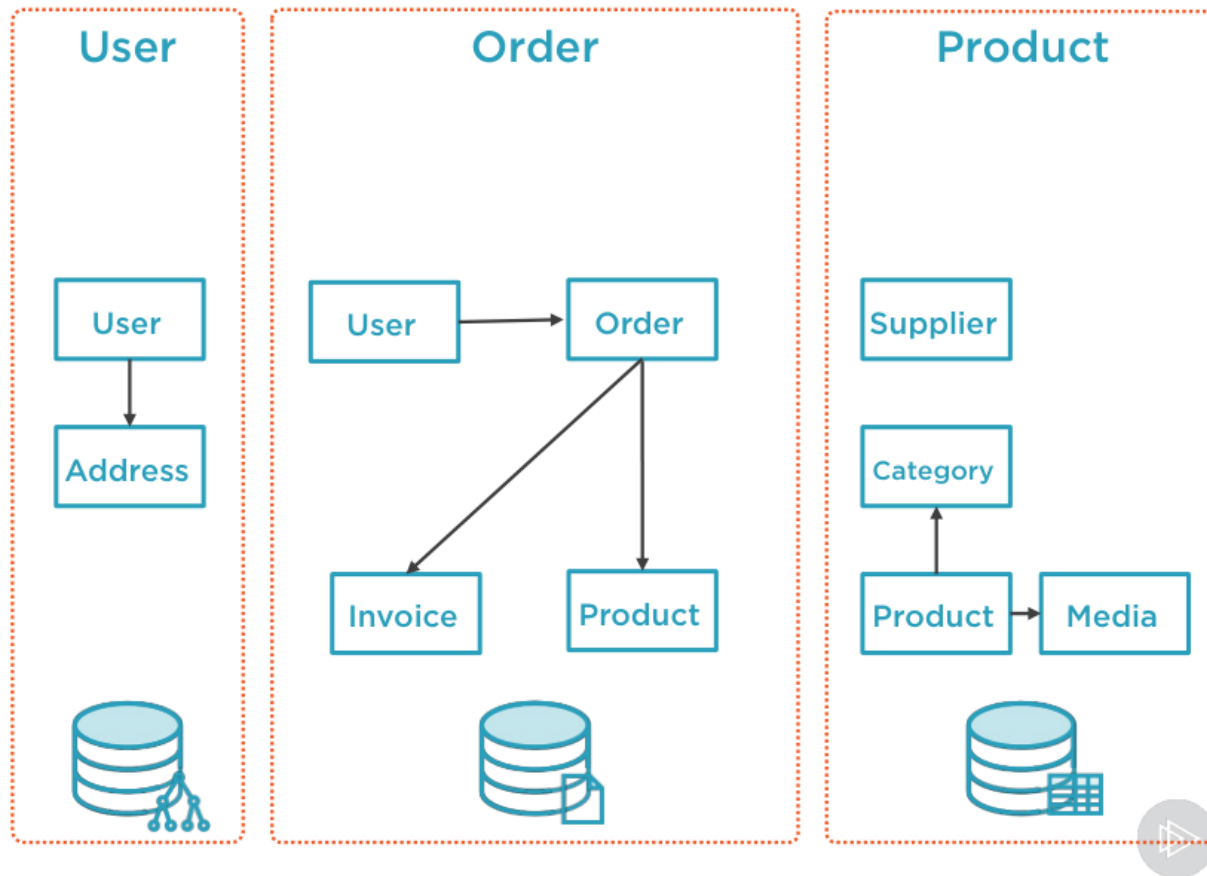
DAO DATA ACCESS OBJECT

ITERATORS

Source: www.iteratorshq.com



Data Synchronization



Causes:

Main reason is Db-per-service – Data decentralized, but it can be network issues (fallacies of distributed computing, next module), race conditions, etc.

Problems:

- BASE instead of ACID, Eventual consistency
- Out-of-order transactions, partial Failure (Update order after a lack of product on inventory)
- Stale data (temp data inconsistency)

Solution:

- Aggregates as consistence limits
- Idempotency, Correlation IDs and Traceability
- Saga Pattern (next section)
- CQRS (next module)
- CDC
- Transactional Outbox (Debezium/CDC or RabbitMQ/Kafka/Broker)

Shared Kernel

“some subset of the domain model that the two teams agree to share. Of course this includes, along with this subset of the model, the subset of code or of the database design associated with that part of the model. This explicitly shared stuff has special status, and shouldn't be changed without consultation with the other team.”

Eric Evans, The Blue Book

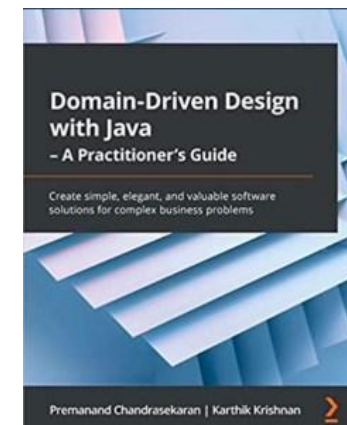
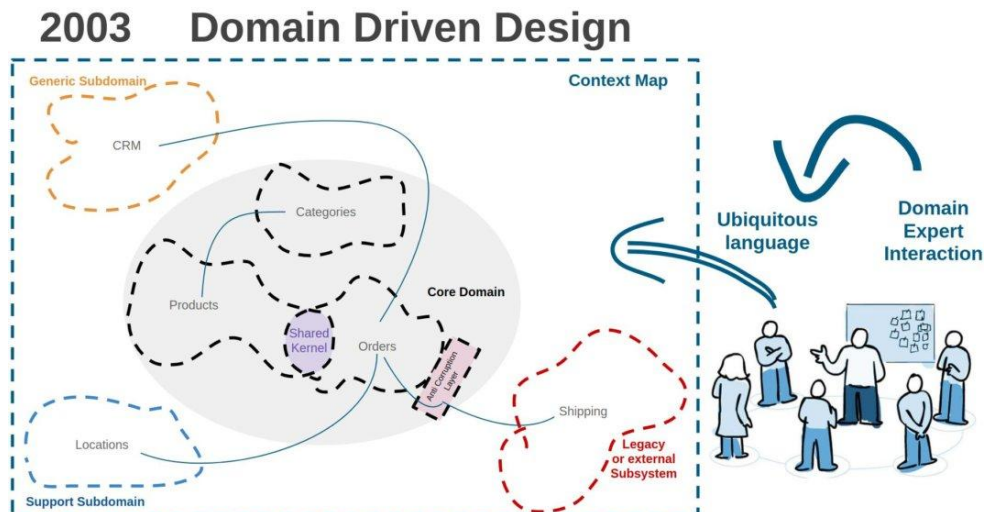
Special case of Context Mapping: Common evolution bounded context through teams. This is an exception to 1:1 relationship, Team and Bounded Context.

Usually Generic/Core subdomains and keep **small**. Same contract to all teams, and don't change frequently.

Any change due to consult/approve for all teams. CI/CD to keep consistency.

Example cases:

Customer on eCommerce/Drone Management



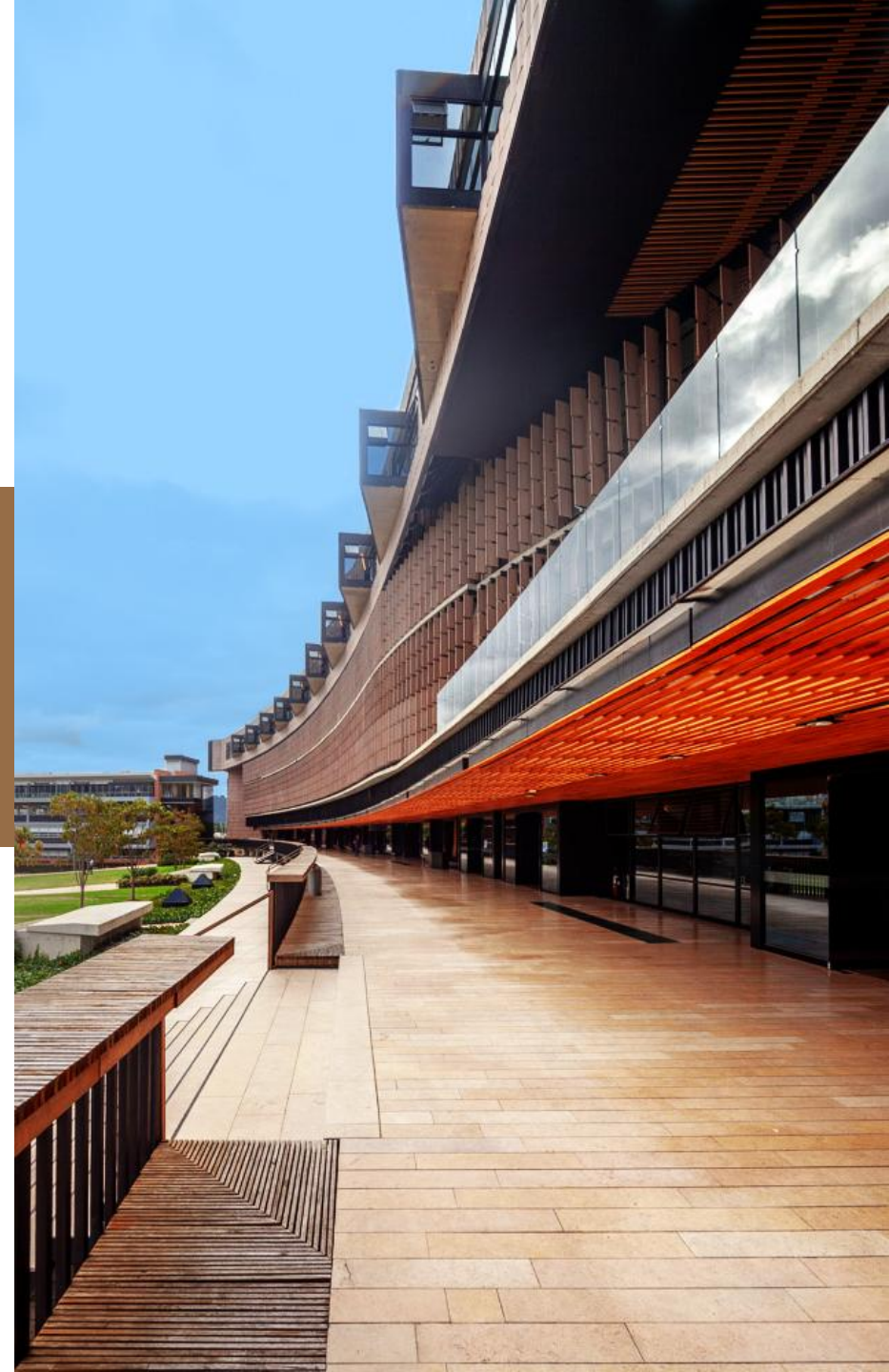
Design Techniques: Assemblage

Change the name after of it. Richardson.



Universidad de
La Sabana

FACULTAD DE INGENIERÍA



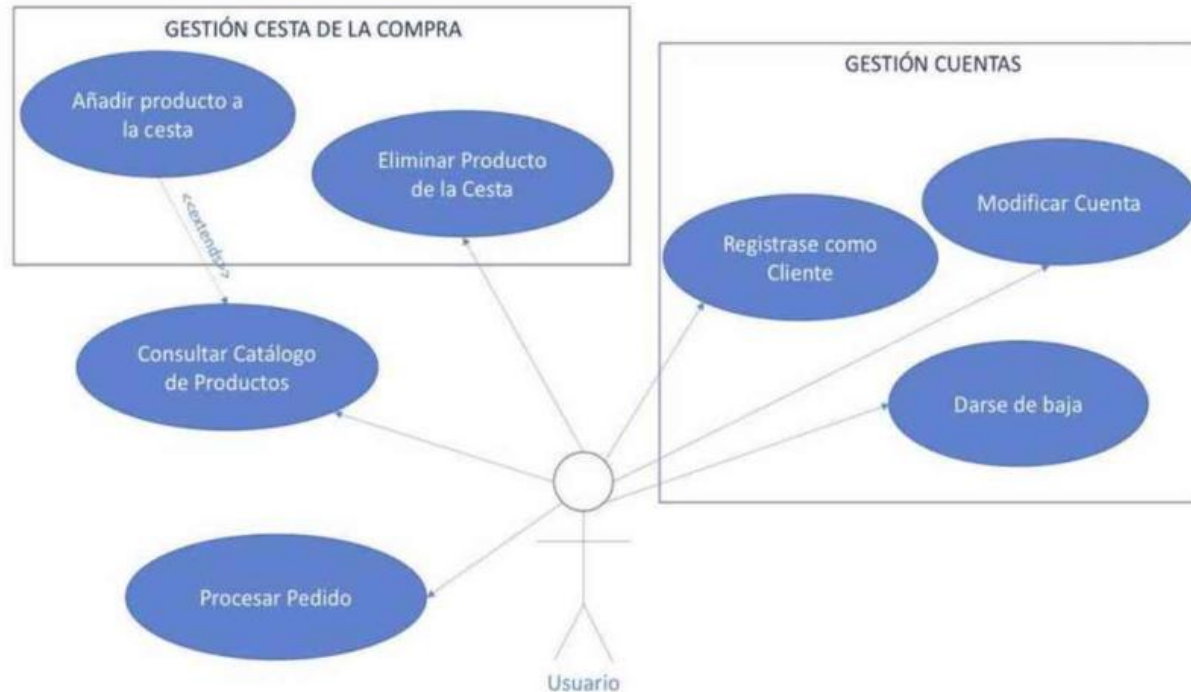


Figura 2.5. Diagrama de casos de uso



Figura 2.6. Descomposición en microservicios basada en sustantivos



Anemic DDD ?
Where are the biz logic? Can
improve over time?

i.e. Cart and Promotion Rules.
Cart and Market Rules.

What is my business value?

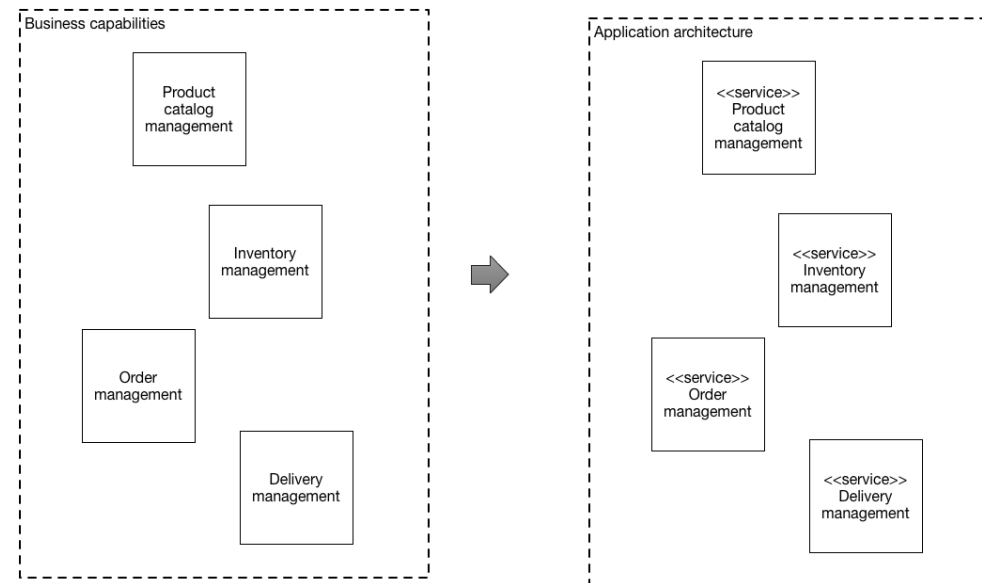
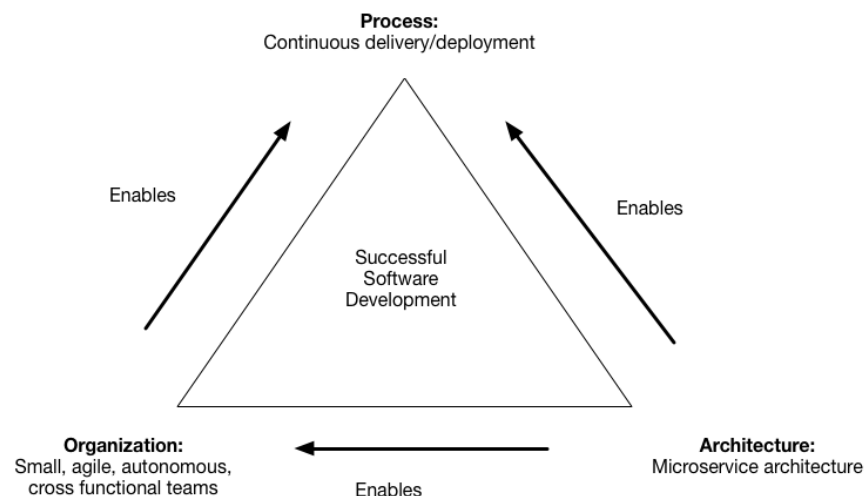
Complementary model

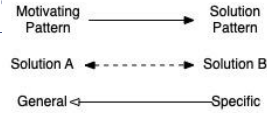
DDD can model: microservices, modular monolith and hexagonal architecture. It's based on biz complexity.

Assemblage can model: microservices mainly. It's based on tech complexity; its basis are distributed systems and autonomous.

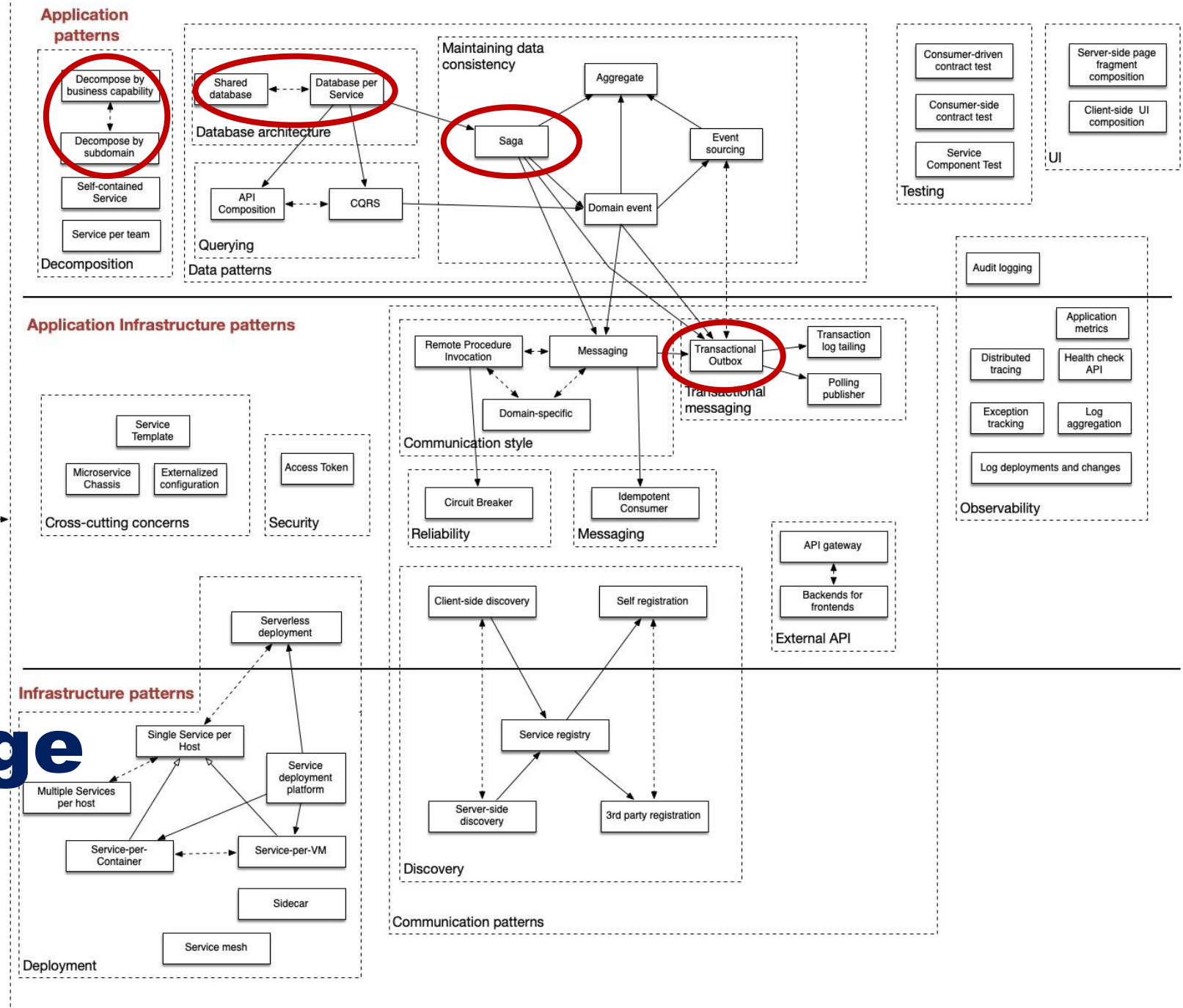
Decomposition based on Biz Capabilities (Biz Add Value) and Subdomains (similar to DDD – UL, Strategic, Tactical).

Data goes to Saga pattern, CQRS and Transactional Outbox (Assemblage) instead of Tactical DDD (DDD).





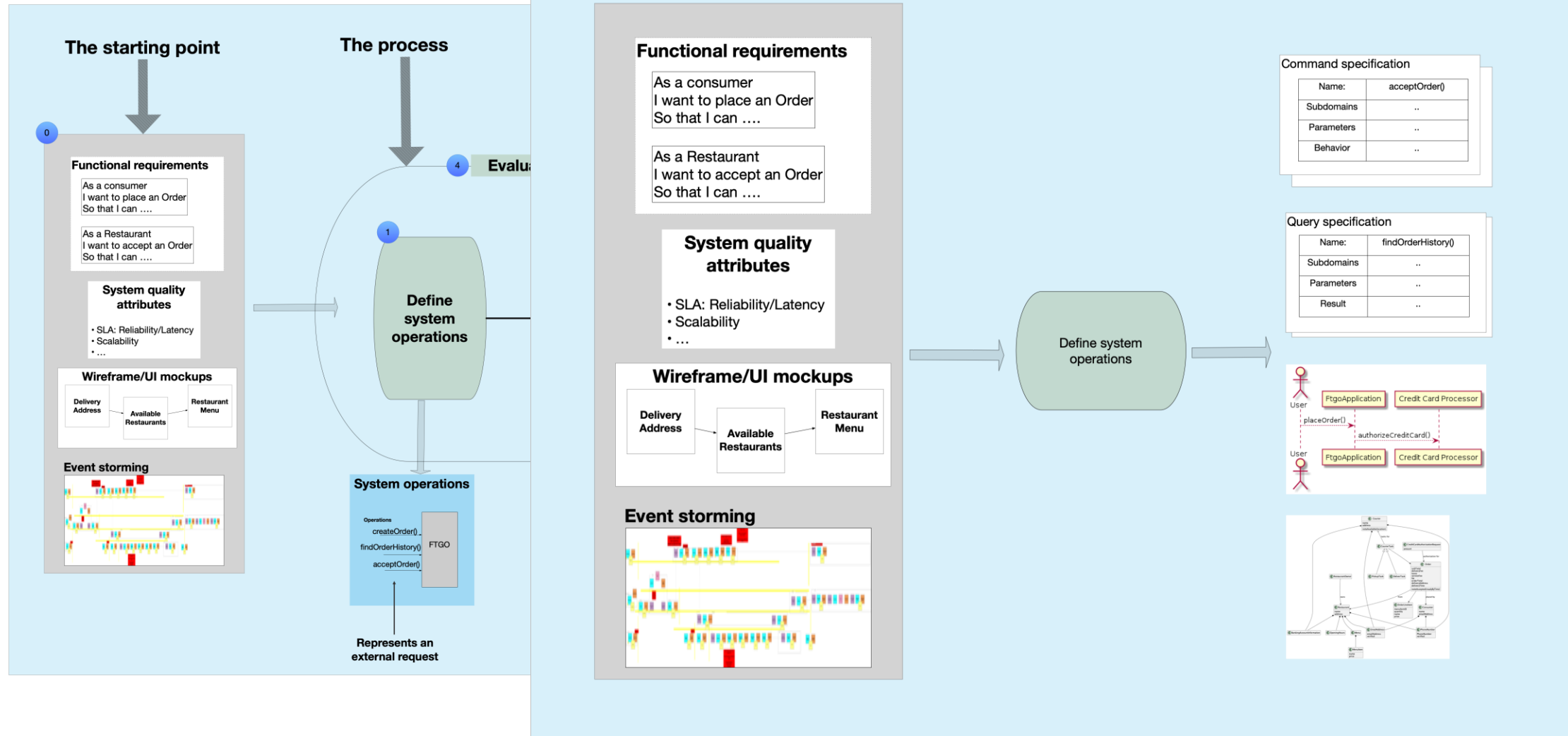
The Microservice Architecture Pattern Language



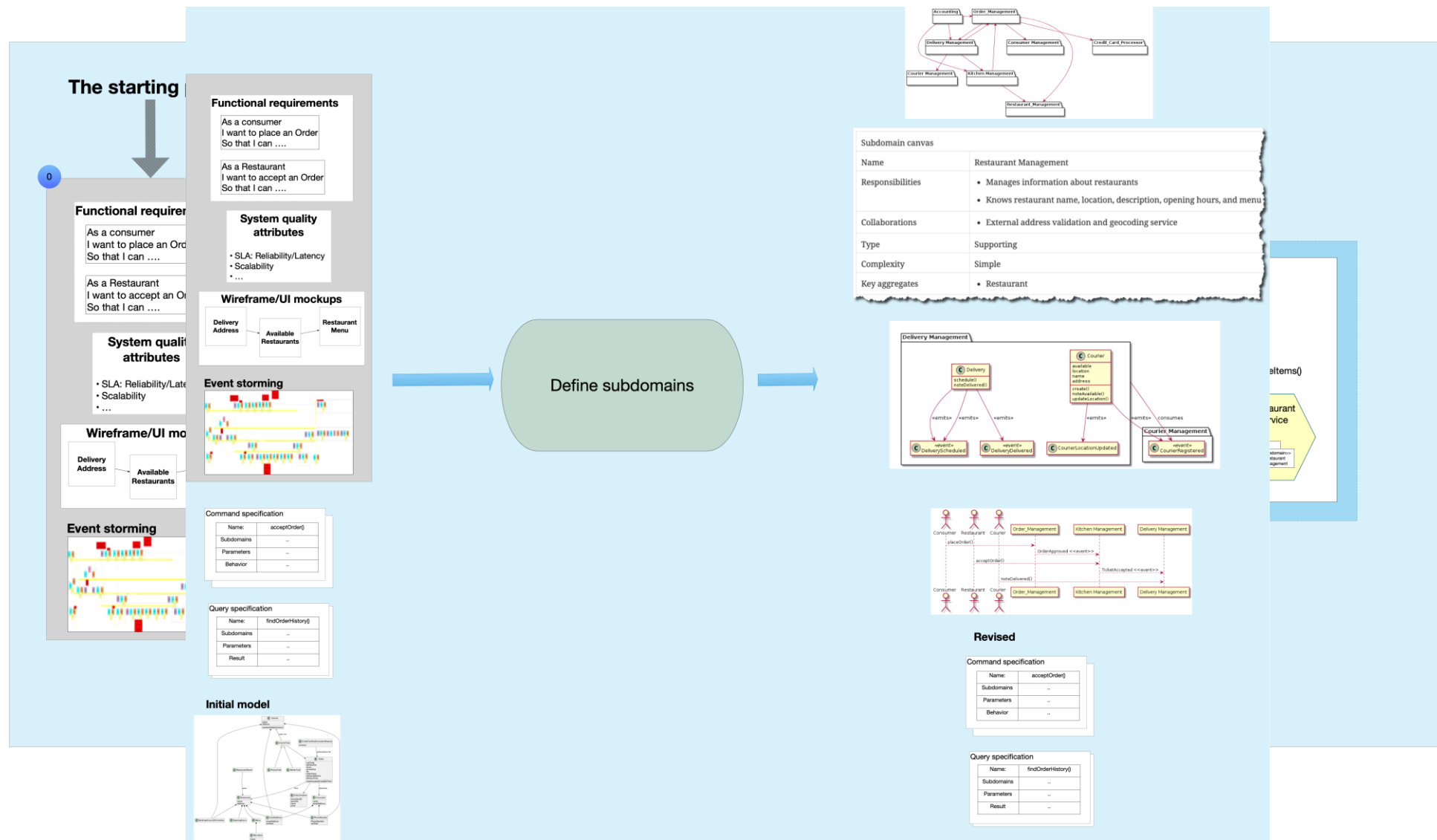
Pure Assemblage

Learn-Build-Assess Pattern
[Link](#)

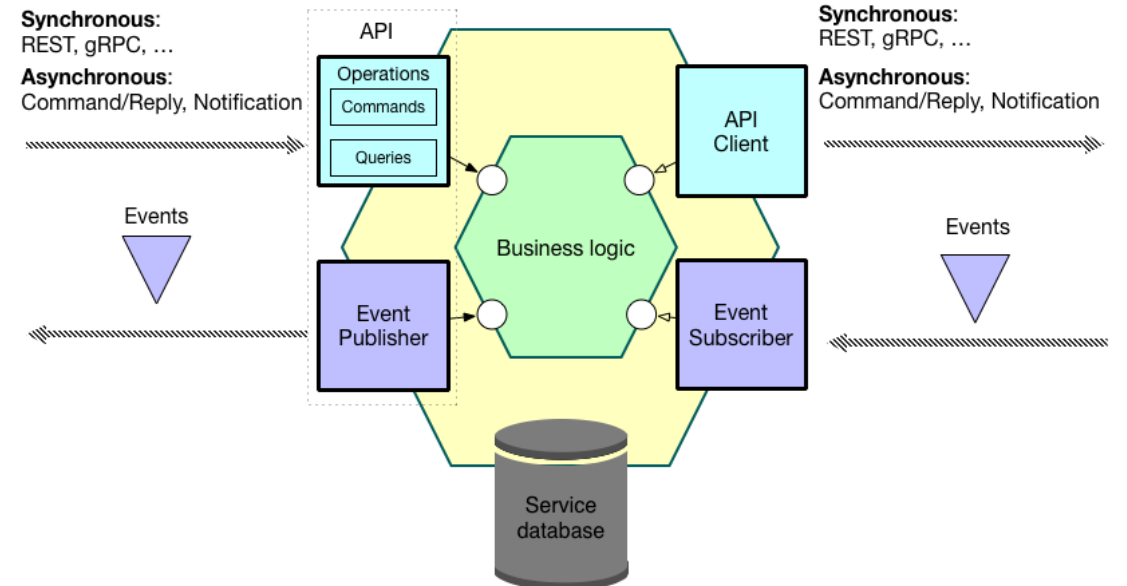
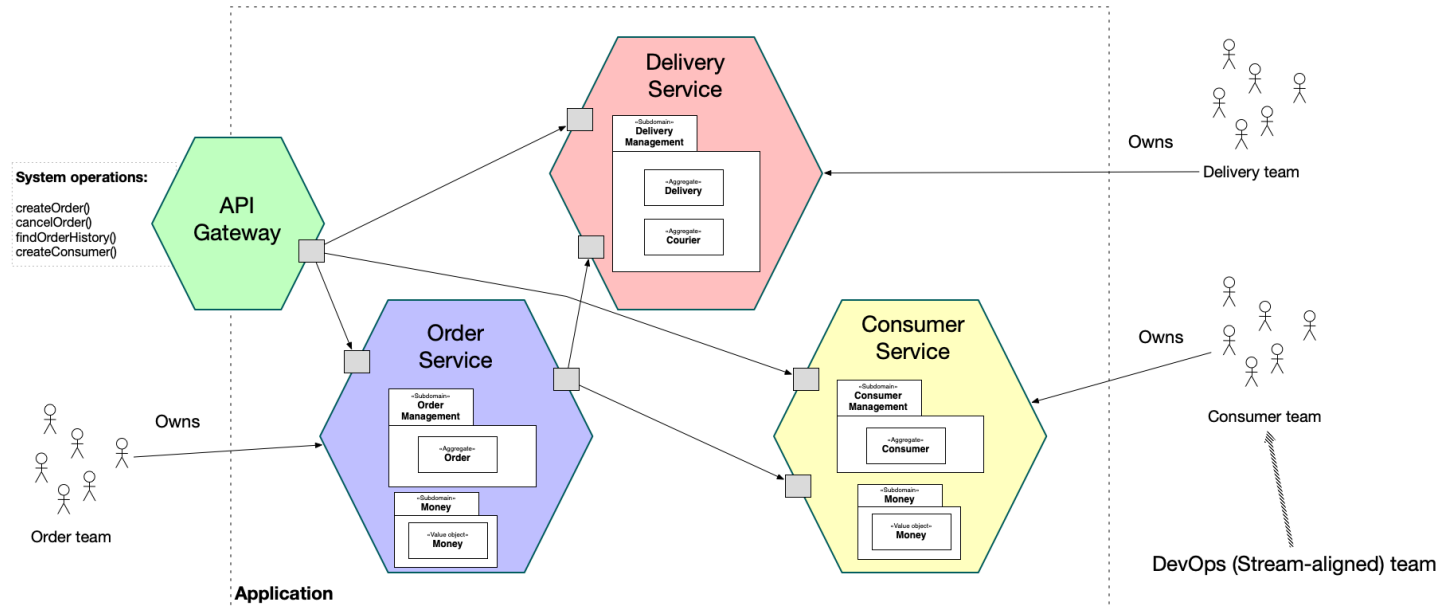
Assemblage Method I



Assemblage Method II



Assemblage Method III

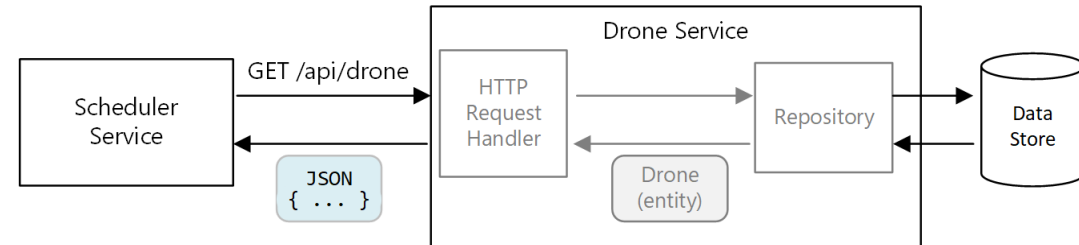
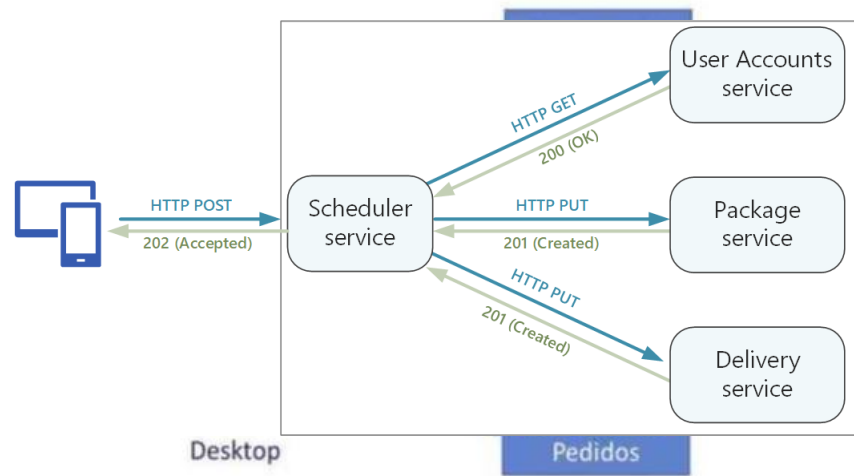




Microservices communication

Direct, through API Gateway and;
Service Discovery and Registry

API Design and Implementation



Operations/Services

Define:

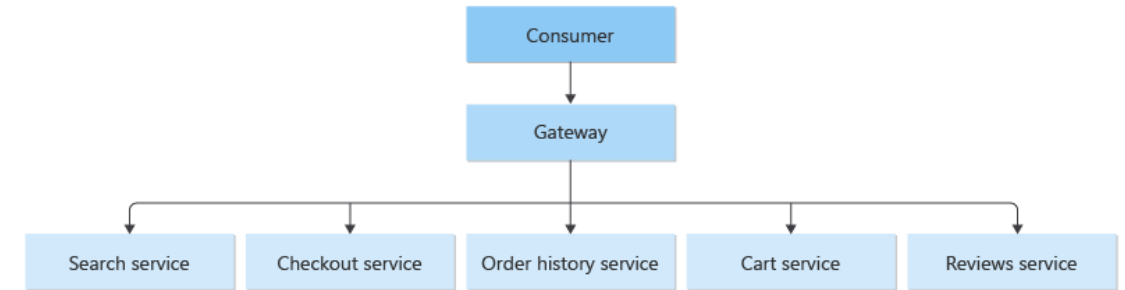
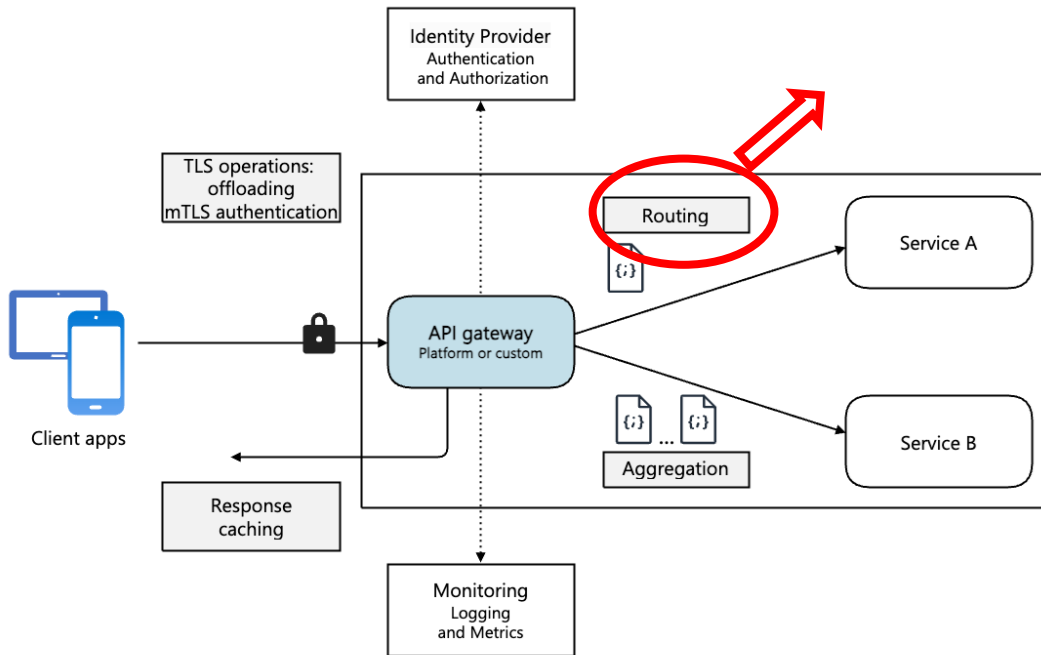
- REST vs RPC, i.e. JSON/YAML/XML over HTTP vs gRPC, Avro, and Thrift.
- Serialization
- WSDL, Swagger, OpenAPI, IDL
- Language Support
- Versioning
- Idempotency

DDD concept	REST equivalent	Example
Aggregate	Resource	<code>{ "1":1234, "status":"pending"... }</code>
Identity	URL	<code>https://delivery-service/deliveries/1</code>
Child entities	Links	<code>{ "href": "/deliveries/1/confirmation" }</code>
Update value objects	PUT or PATCH	<code>PUT https://delivery-service/deliveries/1/dropoff</code>
Repository	Collection	<code>https://delivery-service/deliveries?status=pending</code>

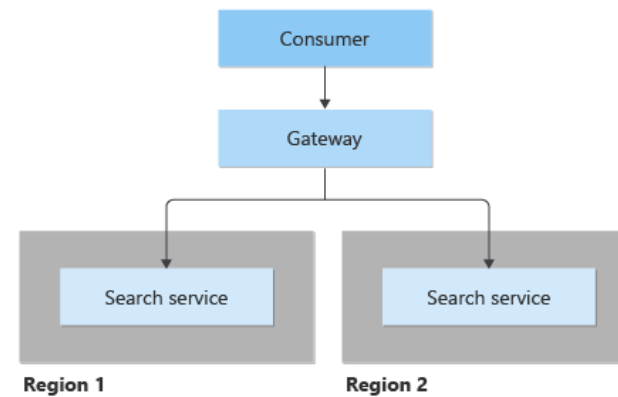
Web API Design and Implementation: [Link](#)

API Gateway Routing

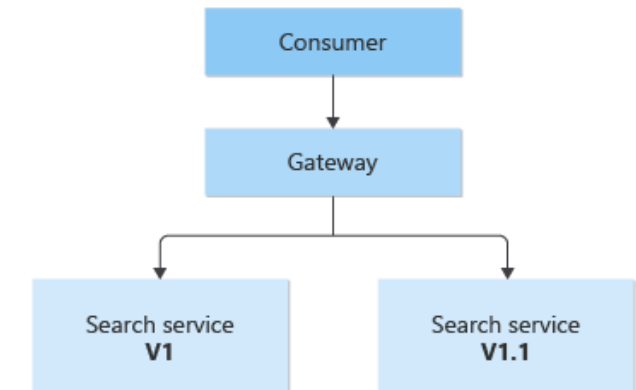
Gateway Routing Pattern



Multiple disparate services

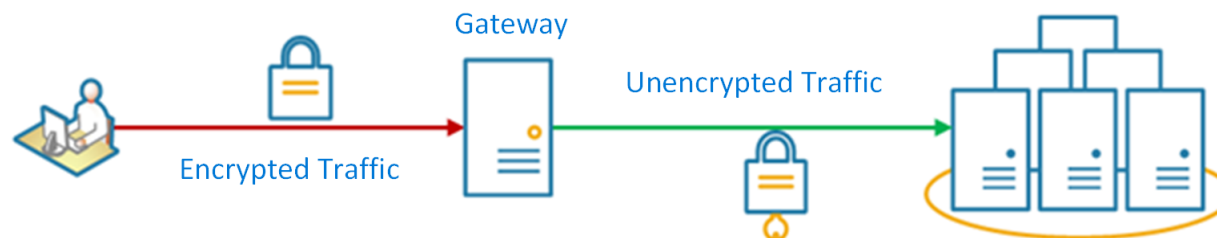
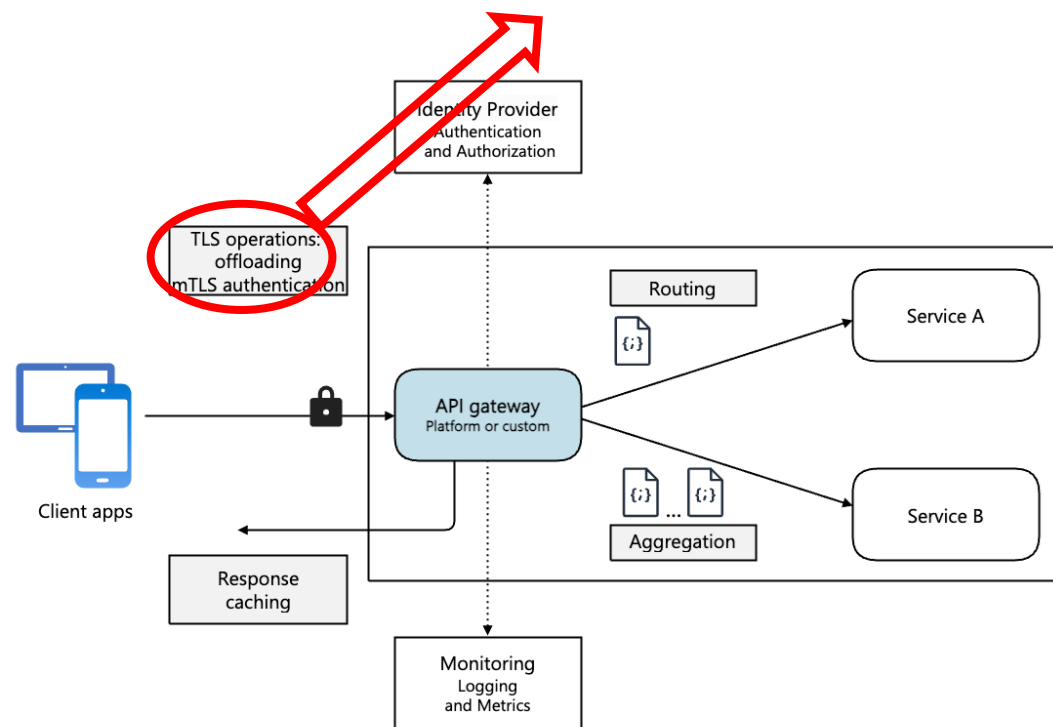


Multiple instances of the same service



Multiple versions of the same service

API Gateway Offloading

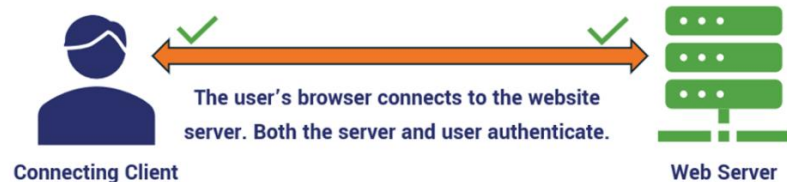


Traditional TLS vs mTLS Connections

Traditional TLS Encryption



Mutual TLS Encryption



Implementation Model

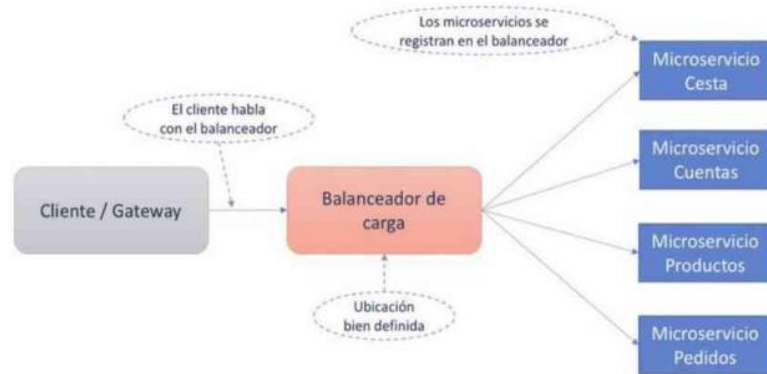


Figura 2.13. Descubrimiento de Server Side

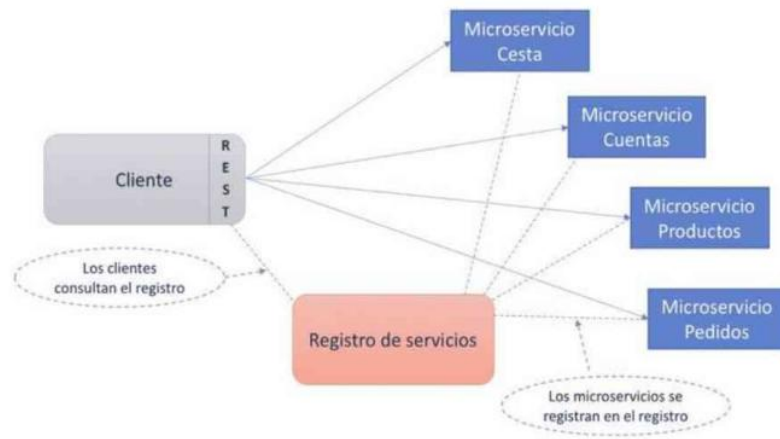


Figura 2.15. Descubrimiento Client Side

Need	MsEI	AWS
Central Configuration	Spring Cloud Config	AWS AppConfig (TF)/Param Store
Service discovery	Netflix Eureka	CoreDNS (EKS)/AWS Cloud Map
Dynamic routing and load balancing	Netflix Ribbon	Ingress Controller (EKS)/ELB/AWS App Mesh
Circuit Breaker	Netflix Hystrix	AWS App Mesh
Monitoring	Netflix Hystrix Dashboard and Turbine	Amazon CloudWatch/AWS X-Ray
Centralized log analysis	ELK	OpenSearch
Edge Server	Netflix Zuul	API GW
Protection APIs based on OAuth standard	Spring Cloud + Spring Security OAuth	API GW+Lambda Auth /Cognito
Microservice	Spring Boot , Spring Cloud Data Flow, Spring Cloud Stream, Librerías comunicación con backends	Compute Services
Asynchronous communication infrastructure	RabbitMQ	Amazon MQ/SQS



Universidad de
La Sabana

FACULTAD DE INGENIERÍA

Migration Strategies

Strangler Fig

Anti-corruption Layer and

Dapr



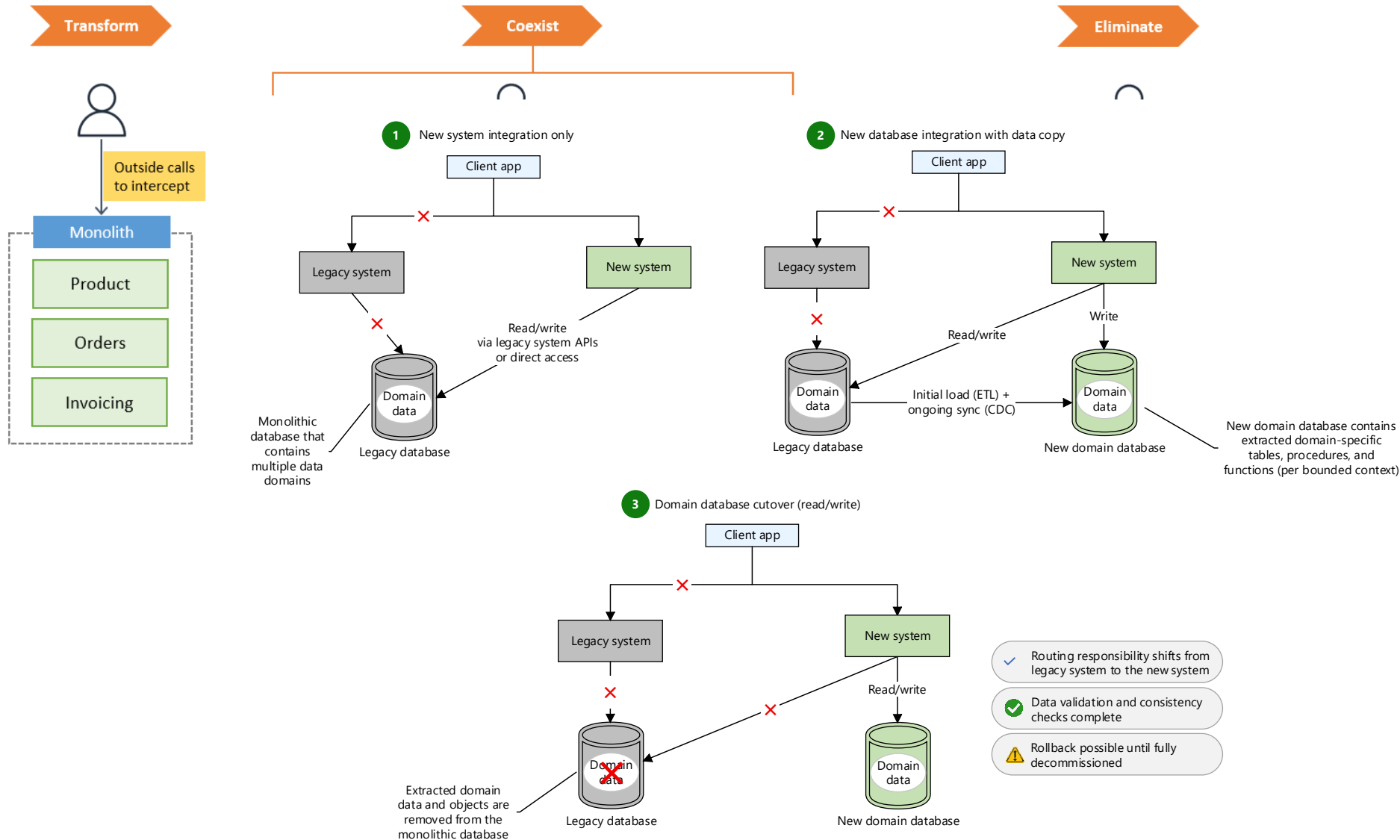
How you estimate a migration ?

The Iron Triangle: Time, Cost (Infra&Talent) and Scope.

Method

Strangler Fig

Azure call
Façade to the
proxy.



Anticorruption Layer

Why ?

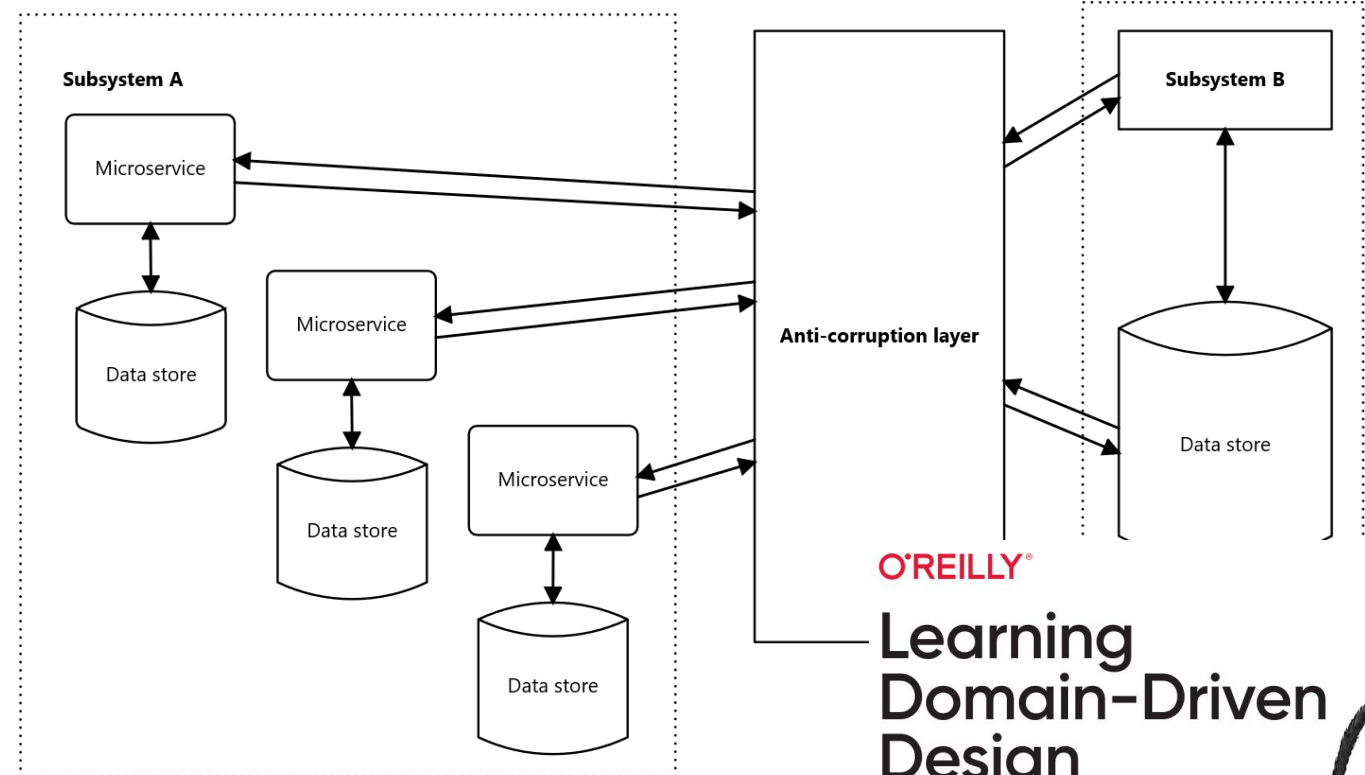
Need to keep a relation between Upstream/Downstream without disruption (Semantic Integrity).

How?

Facades, Adapters, Object Translators.

Use cases:

- Context Mapping
- Migration



Microsoft
Azure

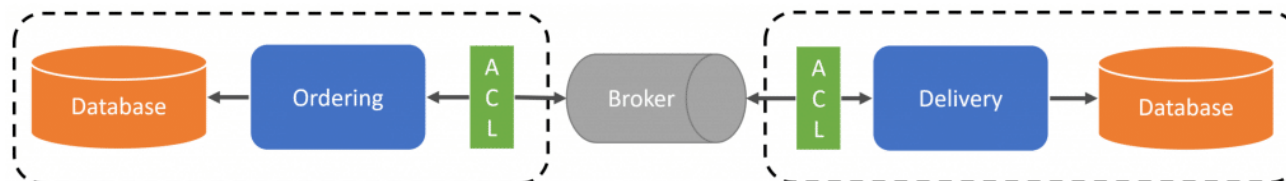
O'REILLY®

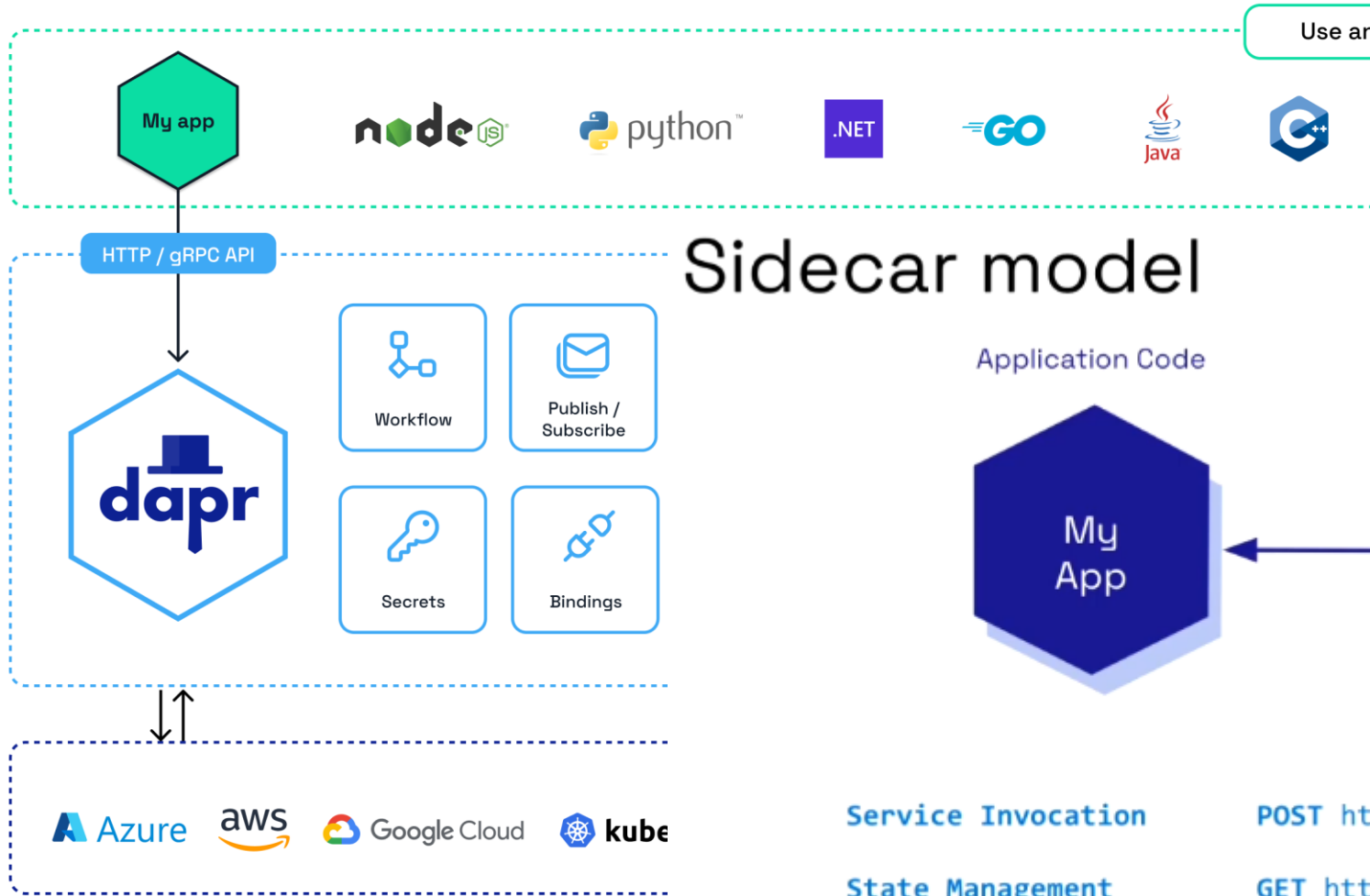
Learning Domain-Driven Design

Aligning Software Architecture and Business Strategy



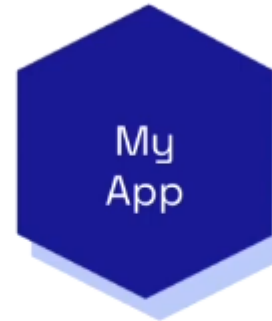
Vlad Khononov
Foreword by Julie Lerman





Sidecar model

Application Code



Service Invocation

State Management

Publish & Subscribe

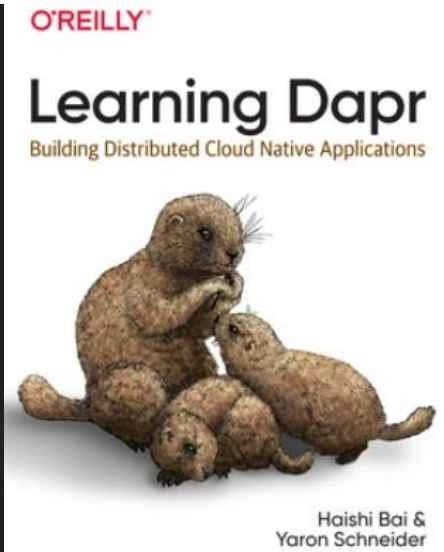
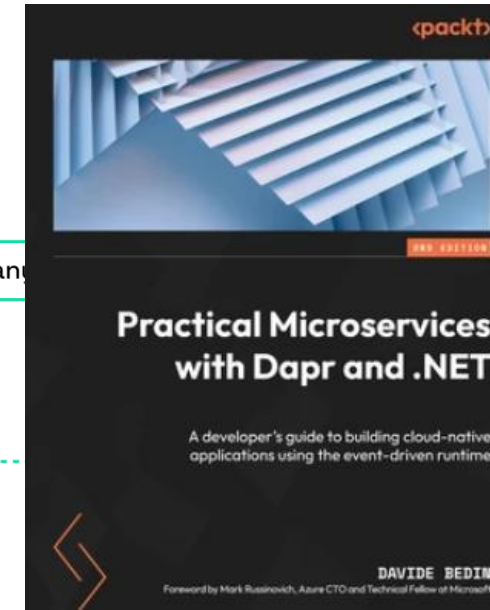
Secrets Management

POST <http://localhost:3500/v1.0/invoke/cart/method/order>

GET <http://localhost:3500/v1.0/state/inventory/item67>

POST <http://localhost:3500/v1.0/publish/order>

GET <http://localhost:3500/v1.0/secrets/vault/password42>



Dapr sidecar

The background of the slide is a photograph of orange flowers with black centers in the foreground, and a multi-story building with a grid of windows in the background. A large dark blue rectangle is overlaid on the right side of the image, containing the title and subtitle. A solid brown vertical bar is on the far right edge.

Additional Patterns

Saga Pattern and Microfrontends

Sagas - Choreography



Figura 7.3. Rollback en una transacción en saga con coreografía

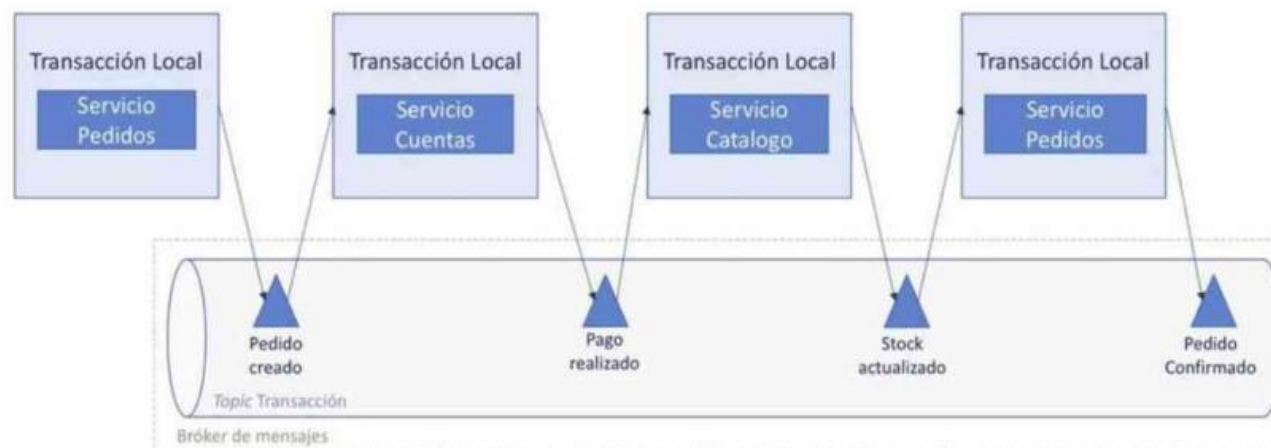


Figura 7.4. Transacción en saga con coreografía y bróker de mensajería

Sagas - Orchestrator

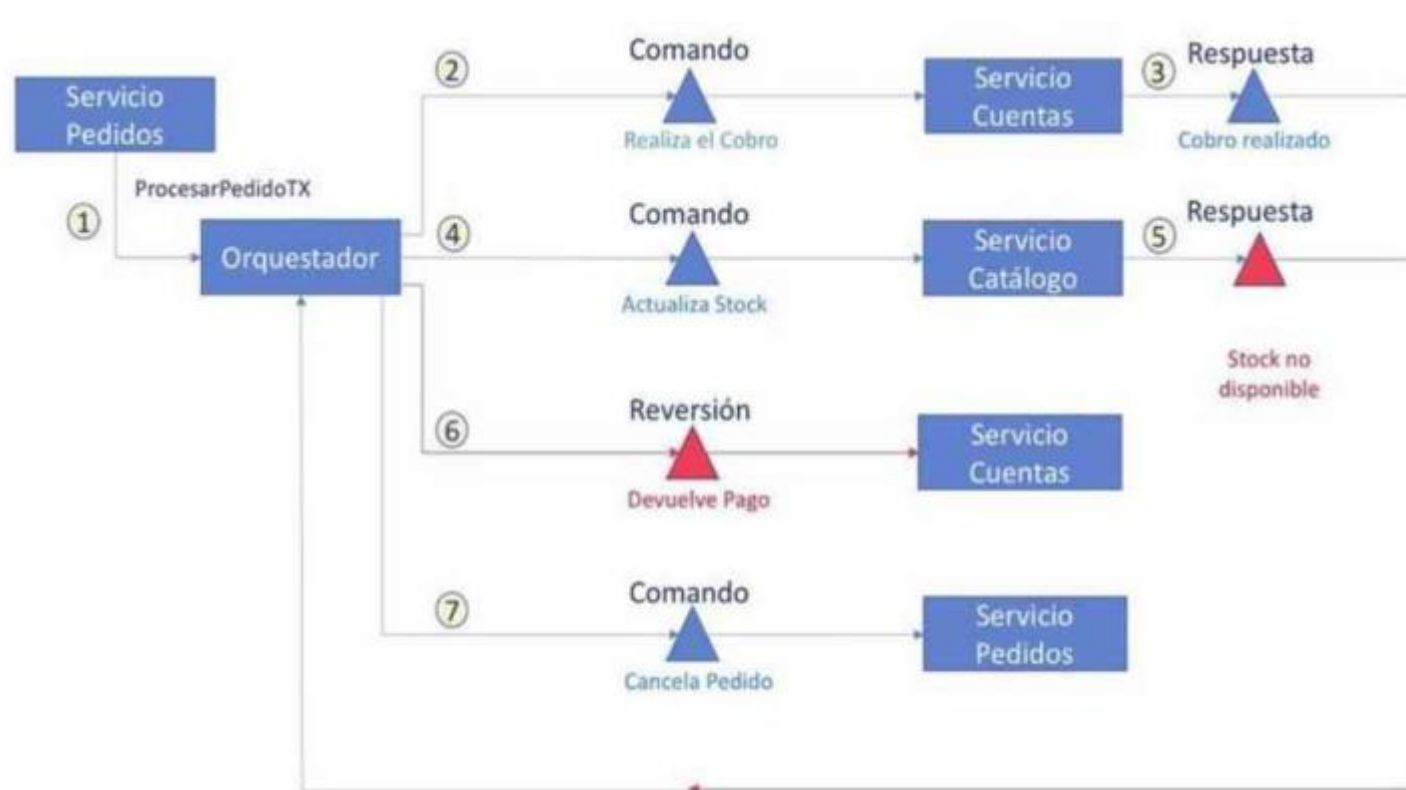


Figura 7.6. Rollback en una transacción en saga con orquestador

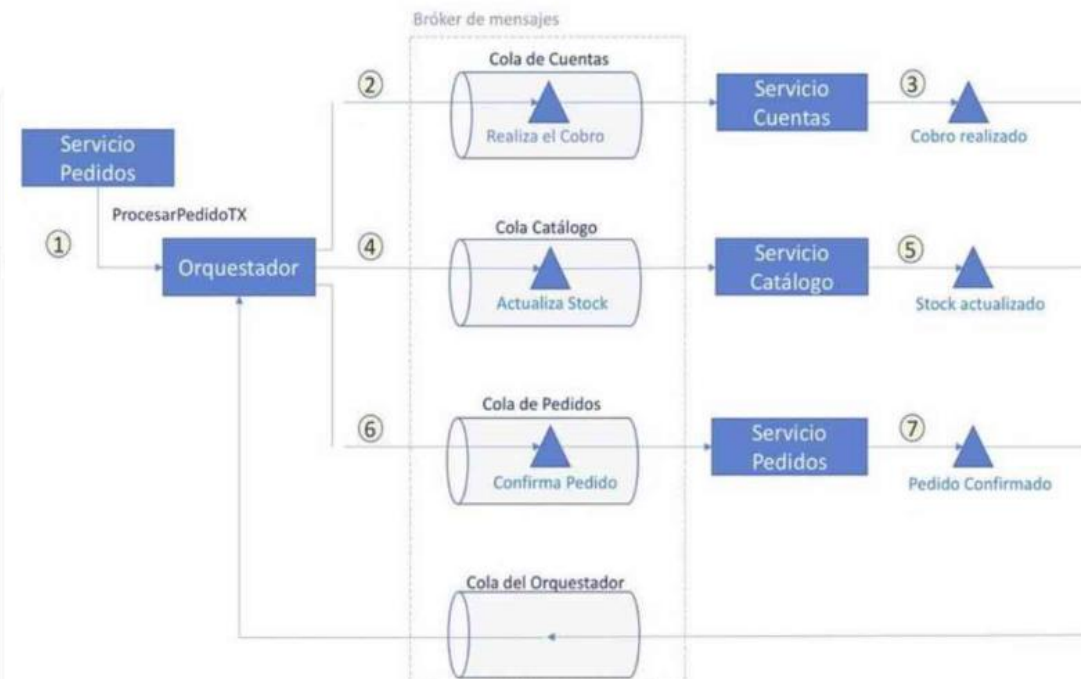
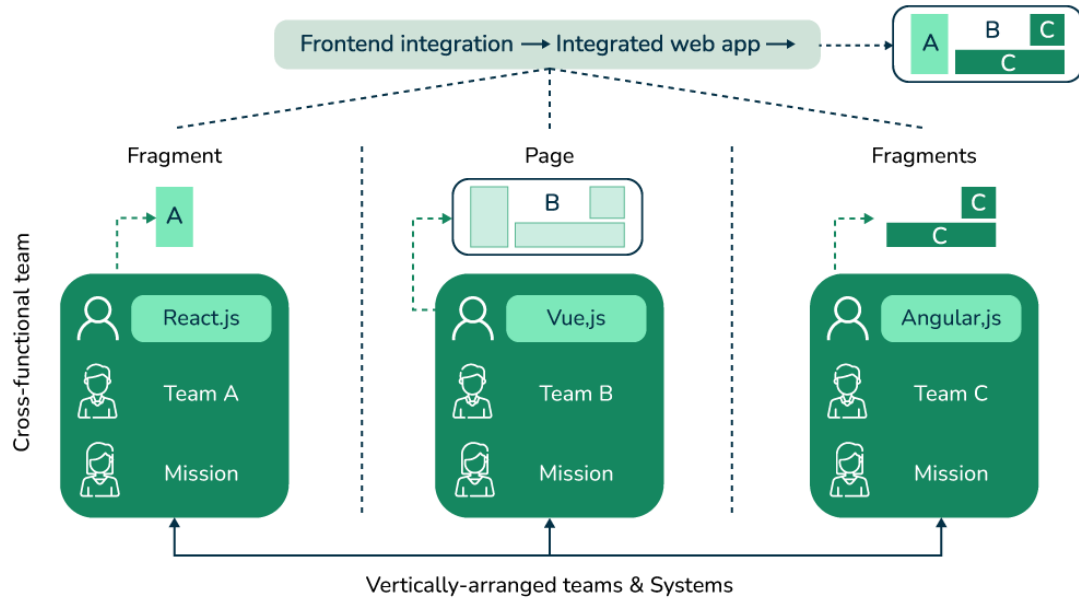


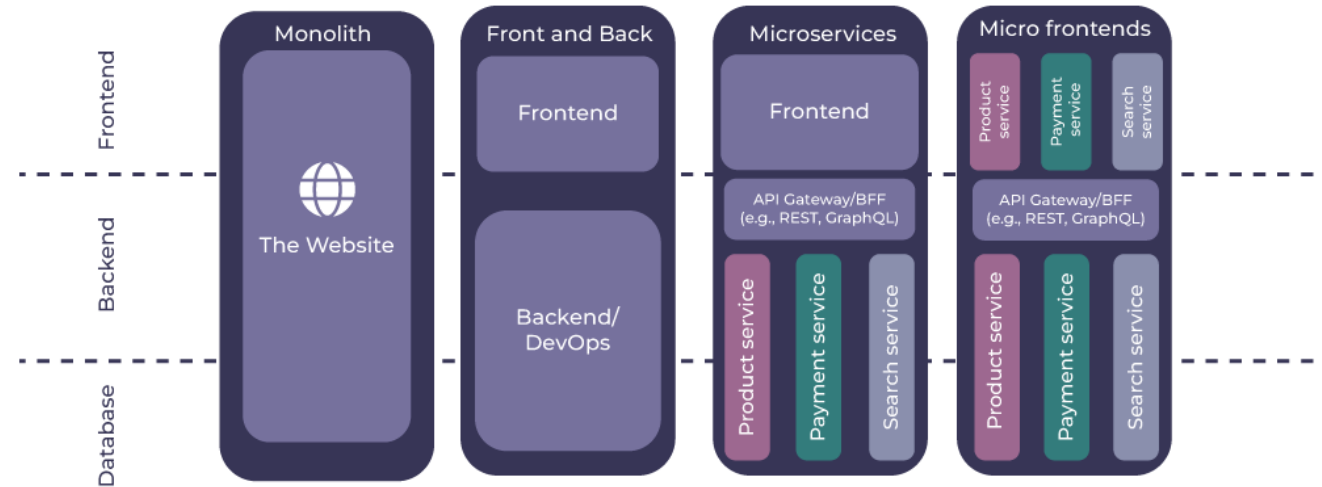
Figura 7.7. Transacción en saga con orquestador y bróker de mensajería

Microfrontends I

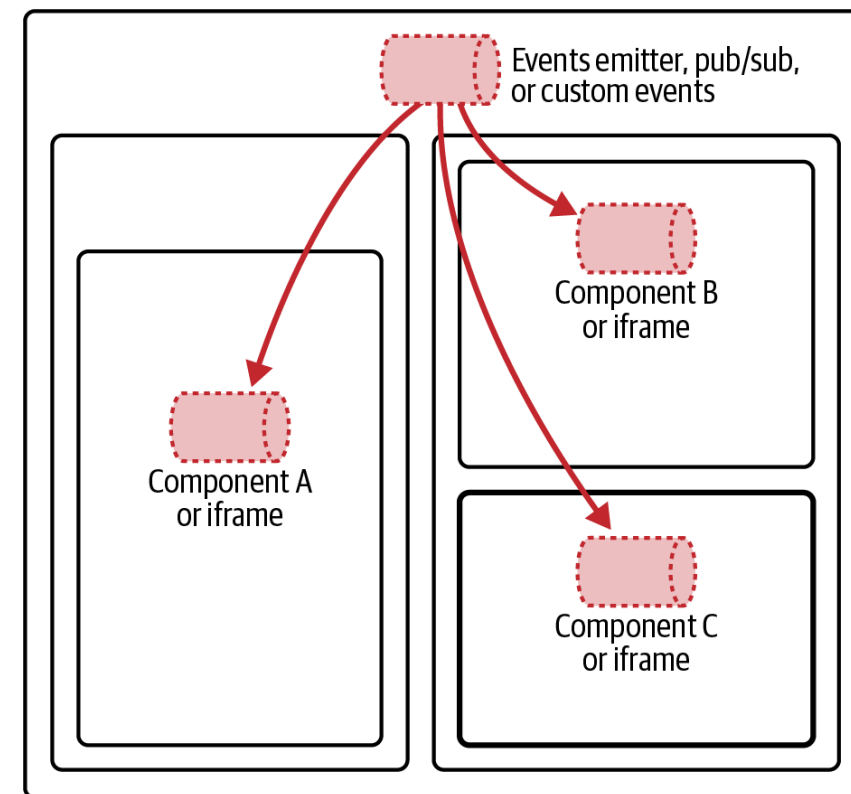
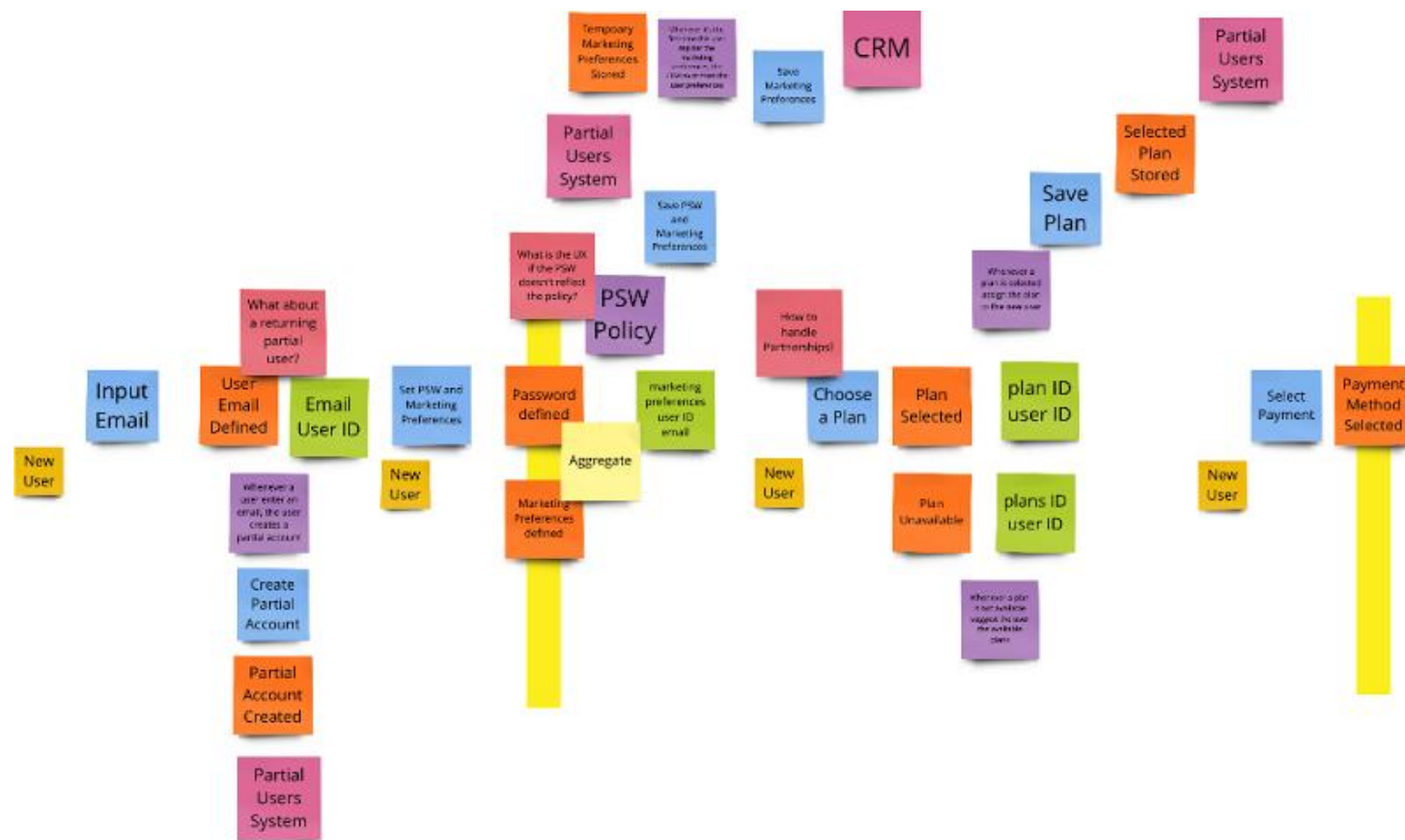
Micro Frontend Architecture



DIFFERENT ARCHITECTURAL APPROACHES



Microfrontends II



Options –At least, that I explore-:

- DOM Options
- Local Storage

References

- [MSPa] Pluralsight. (n.d.). Microservices architecture [Skill path]. <https://app.pluralsight.com/paths/skill/microservices-architecture-new>
- [AzMs] Microsoft. (n.d.). Microservices architecture style. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
- [MsEi] Roldán Martínez, D., & Valderas Aranda, P. J. (2018). *Microservicios: Un enfoque integrado*. Rama Editorial.
- [MFAm] Fowler, M. (2003, November 25). *Anemic domain model*. MartinFowler.com. <https://martinfowler.com/bliki/AnemicDomainModel.html>
- [AssD] Richardson, C. (2023). The Assemblage architecture definition process. Microservices.io. <https://microservices.io/post/architecture/2023/02/09/assemblage-architecture-definition-process.html>
- [AzAD] Microsoft Azure. (n.d.). Designing APIs for microservices. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/architecture/microservices/design/api-design>
- [AwSF] Amazon Web Services. (n.d.). Strangler Fig pattern. AWS Prescriptive Guidance. <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-decomposing-monoliths/strangler-fig.html>
- [AzSF] Microsoft Azure. (n.d.). Strangler Fig pattern. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>
- [MFe] Mezzalana, Luca (2025). Building Micro-Frontends, 2nd Edition. O'Reilly. <https://learning-oreilly-com.ez.unisabana.edu.co/library/view/building-micro-frontends-2nd/9781098170776/>



Universidad de
La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co